

Minimize Malware Spread

© Authored by Arpit Raj, LNMIIT Jaipur | August 5, 2026

LeetCode 924

Salesforce

DropBox

DSU

Problem Walkthrough

```
[ 0 -- 1 -- 2 ]  
[ 3 -- 4 ]
```

Initial: (1, 3)

- Then if we give vaccine to **1** then whole component of size 3 gets saved.
- If we give vaccine to the **3**, the whole component of size 2 gets saved.

Hence giving to **1** is better as it minimizes, so o/p = 1.

Key Observations (Obv)

Critical Insight

If a component has **more than 1 infected**, it can never be saved by just 1 removal, hence focus on other components.

So we need: leader → component size → no. of infected nodes

- Component size is already there in `ds.size[]`.
- For no. of infected nodes we can maintain an `unordered_map`.
- If infected count == 1, removing it saves component size. Otherwise, 0 save.
- If tie in max saved component, **pick smallest index**.

C++ Solution

Disjoint Set Union (DSU) Implementation

```
class dsu{
public:

    vector<int> parent;
    vector<int> size;
    dsu(int n){

        parent.resize(n);
        size.resize(n,1);
        for(int i=0;i<n;i++){
            parent[i]=i;
        }

    }

    int findpar(int node){

        if(parent[node]==node){
            return node;
        }

        parent[node]=findpar(parent[node]);
        return parent[node];
    }

    void merge(int u,int v){

        int pu=findpar(u);
        int pv=findpar(v);

        if(pu==pv) return;

        if(size[pu]<size[pv]) swap(pu,pv);
        size[pu]+=size[pv];
        parent[pv]=pu;
    }

};

class Solution {
public:
    int minMalwareSpread(vector<vector<int>>& graph, vector<int>& initial) {

        int n=graph.size();
        dsu ds(n);
```

```

for(int i=0;i<n;i++){
    for(int j=i+1;j<n;j++){

        if(graph[i][j]==1)
            ds.merge(i,j);
    }
}

//har component me kitne infected he ye chahiye hoga
//agar 1 se zyada infected hue to nhi bacha skte
unordered_map<int,vector<int>> infcnt;

for(auto &node:initial){
    int boss=ds.findpar(node);
    infcnt[boss].push_back(node);
}

//ek vector to store {savesize,idx removed}
vector<pair<int,int>> finder;

for(auto &it:infcnt){
    int boss=it.first;
    vector<int> infs=it.second;

    if(infs.size()==1){ //1 hi infected hua to hi bach paayega
        finder.push_back({ds.size[boss],infs[0]});
    }
}

sort(finder.begin(),finder.end(),[](auto &a,auto &b){

    if(a.first!=b.first){
        return a.first>b.first; //largest component size saved first
    }

    return a.second<b.second; //phir if tie then smallest index
});

if(finder.empty()) {
    sort(initial.begin(),initial.end());
    return initial[0];
} //koi bhi 1 infected wala nhi mila to koi sa bhi smallest remove kr
denge

return finder[0].second;

}
};

```